



Research-backed recommendations for Phase 2: Operation model + node simulation

Your proposed design is solid. Below are targeted refinements based on current best practices for SQLite patterns, Go DB usage, and event-sourced systems. These changes will make your implementation more efficient, flexible, and production-aligned without adding complexity.

1. Operation write pattern: Append-only + INSERT OR IGNORE (not upsert)

Key insight

For an event log, you want **pure append-only** semantics:

- Each `OperationID` is generated once and never updated.
- Duplicates (retries) should be **silently ignored**, not updated.
- You never want `INSERT OR REPLACE` or `ON CONFLICT DO UPDATE` for the operation log — those imply mutation, which breaks event-sourcing immutability. ^{[1] [2] [3] [4] [5]}

Recommended pattern

Table schema:

```
CREATE TABLE counter_operations (  
  operation_id TEXT PRIMARY KEY,  
  counter_id TEXT NOT NULL,  
  node_id TEXT NOT NULL,  
  epoch INTEGER NOT NULL,  
  sequence INTEGER NOT NULL,  
  amount INTEGER NOT NULL,  
  created_at TEXT NOT NULL  
);  
  
-- Optional: index for node-level queries  
CREATE INDEX idx_ops_node_epoch ON counter_operations(node_id, epoch);
```

Insert function:

```
func InsertOperation(db *sql.DB, op model.Operation) (inserted bool, err error) {  
  _, err = db.Exec(`  
    INSERT OR IGNORE INTO counter_operations (  
      operation_id, counter_id, node_id, epoch, sequence, amount, created_at
```

```

        ) VALUES (?, ?, ?, ?, ?, ?, ?)
    `, op.OperationID, op.CounterID, op.NodeID, op.Epoch, op.Sequence, op.Amount, op

    if err != nil {
        return false, err
    }

    // Check if row was actually inserted (vs ignored as duplicate)
    // Option 1: use changes() in same transaction
    // Option 2: query existence before insert (slower but clearer)
    return true, nil
}

```

Why INSERT OR IGNORE:

- **Fast:** No lookup before write; SQLite checks the unique constraint and skips if duplicate. ^[6]
^[7] ^[8]
- **Safe:** Existing row is never modified; duplicates are dropped silently. ^[3] ^[9] ^[6]
- **Matches event-sourcing:** Operations are immutable; you never update an event. ^[10]

Avoid:

- **INSERT OR REPLACE:** Deletes the old row and inserts a new one, which:
 - Resets columns you didn't specify.
 - Can break foreign keys or triggers.
 - Is semantically wrong for an immutable event log. ^[2] ^[4] ^[5] ^[11] ^[1] ^[3]
- **ON CONFLICT DO UPDATE:** Implies mutation; not appropriate for events. ^[8] ^[12] ^[1]

When upsert *is* appropriate

Upsert (ON CONFLICT DO UPDATE) is for **derived state** (e.g., manifests, aggregates), not for the event log itself. ^[10]

You'll use it later for:

- Node manifests (update local_value, processed_ranges on conflict).
- Maybe a materialized counter cache.

But for counter_operations, stick with **append-only** + INSERT OR IGNORE.

2. Dual-write problem: Use outbox pattern, not naive dual-write

The issue

Your current plan:

ApplyDelta writes to **both node DB and coordinator DB** (dual-write).

This is the classic **dual-write problem**: two independent writes that can partially fail, leaving the system in an inconsistent state. ^[13]

For a hackathon MVP, a naive dual-write is acceptable if you:

- Treat the **coordinator DB** as **authoritative**.
- Accept that a rare failure between the two writes will be corrected by reconciliation.

But you can make this more robust with minimal extra code using the **transactional outbox pattern**. ^[14] ^[13]

Recommended pattern (lightweight outbox)

Node DB schema (outbox table):

```
CREATE TABLE operation_outbox (
  operation_id TEXT PRIMARY KEY,
  counter_id TEXT NOT NULL,
  node_id TEXT NOT NULL,
  epoch INTEGER NOT NULL,
  sequence INTEGER NOT NULL,
  amount INTEGER NOT NULL,
  created_at TEXT NOT NULL,
  synced INTEGER DEFAULT 0 -- 0 = not yet synced to coordinator, 1 = synced
);

CREATE INDEX idx_outbox_synced ON operation_outbox(synced);
```

Flow:

1. ApplyDelta (within a single transaction on node DB):

- Insert into counter_operations (node-local log).
- Insert into operation_outbox with synced = 0.
- Update in-memory counter.
- Commit.

2. Background sync goroutine (per node or shared):

- Periodically query operation_outbox WHERE synced = 0.
- For each row:
 - Insert into coordinator DB (INSERT OR IGNORE).
 - On success, mark synced = 1 in node DB.
- This can be best-effort; reconciliation will fix any gaps.

Why this helps:

- **Atomicity at the node:** The operation is durably recorded in the node DB before being considered “applied.”
- **Resilience:** If coordinator write fails, the outbox row remains and will be retried.

- **Clear semantics:** Node DB = “what I believe I did”; Coordinator DB = “authoritative global log.”

For the MVP, you can simplify:

- Skip the background goroutine.
- On ApplyDelta, write to node DB first, then coordinator DB.
- If coordinator write fails, log the error but don’t roll back the node write.
- Reconciliation will detect and repair any divergence.

This is a **pragmatic middle ground**: not a full outbox, but better than silent dual-write failure.

3. Go `sql.DB` usage: One `*sql.DB` per file, tuned connection pool

Key insight

In Go’s database/sql:

- `*sql.DB` is a **connection pool**, not a single connection. ^[15] ^[16]
- You should **share** a `*sql.DB` per database file across goroutines.
- Do **not** open/close a new `*sql.DB` per operation.

Recommended pattern

Per-node DB:

```
// In simulation.go, when creating a node:
nodeDB, err := sql.Open("sqlite", dbPath) // e.g., "../data/node-0.db"
if err != nil {
    return nil, err
}

// Tune the pool (SQLite is file-based, so limit concurrency)
nodeDB.SetMaxOpenConns(1)      // SQLite writes are serialized anyway
nodeDB.SetMaxIdleConns(1)
nodeDB.SetConnMaxLifetime(0)   // No need to expire connections for embedded SQLite
```

Coordinator DB:

```
coordDB, err := sql.Open("sqlite", coordDBPath) // e.g., "../data/coordinator.db"
if err != nil {
    return nil, err
}

// Allow a few concurrent reads; writes will still be serialized by SQLite
coordDB.SetMaxOpenConns(4)
coordDB.SetMaxIdleConns(2)
coordDB.SetConnMaxLifetime(5 * time.Minute) // Periodic refresh is fine
```

Why:

- **SQLite write serialization:** SQLite allows only one writer at a time per file. Setting `MaxOpenConns=1` for node DBs avoids contention. ^[16]
- **Coordinator reads:** Multiple goroutines may read the coordinator DB concurrently; a small pool helps.
- **No CGO:** Using `modernc.org/sqlite` (pure Go) means you don't need a C compiler, but the `*sql.DB` pooling principles still apply. ^{[15] [16]}

Avoid:

- Opening a new `*sql.DB` per operation (high overhead).
- Using a single `*sql.DB` for multiple database files (each file needs its own pool).

4. Sequence generation: Per-node, per-epoch monotonic counter

Key insight

Your sequence field should be:

- **Monotonic per (node_id, epoch).**
- **Generated atomically** to avoid gaps or duplicates under concurrency.

Recommended pattern

In Node struct:

```
type Node struct {
    ID          string
    State       model.NodeState
    DB          *sql.DB
    CoordDB     *sql.DB
    mu          sync.Mutex
    counterID   string
    epoch       int64
    nextSeq     int64 // in-memory monotonic counter
}
```

In NewNode:

```
// Fetch max sequence from DB for this node+epoch
var maxSeq sql.NullInt64
err := node.DB.QueryRow(`
    SELECT MAX(sequence) FROM counter_operations WHERE node_id = ? AND epoch = ?
`, node.ID, node.epoch).Scan(&maxSeq)

if err != nil {
    return nil, err
}
```

```

node.nextSeq = 1
if maxSeq.Valid {
    node.nextSeq = maxSeq.Int64 + 1
}

```

In ApplyDelta:

```

func (n *Node) ApplyDelta(amount int64) (*model.Operation, error) {
    n.mu.Lock()
    defer n.mu.Unlock()

    op := &model.Operation{
        OperationID: uuid.New().String(),
        CounterID:   n.counterID,
        NodeID:     n.ID,
        Epoch:      n.epoch,
        Sequence:   n.nextSeq,
        Amount:     amount,
        CreatedAt:  time.Now(),
    }
    n.nextSeq++

    // Insert into node DB, then coordinator DB
    // ...
}

```

Why:

- **Monotonicity:** Guarantees sequence strictly increases per node+epoch.
- **Crash recovery:** On restart, recompute nextSeq from MAX(sequence) in DB.
- **Simplicity:** No need for DB-side auto-increment; Go controls the sequence.

5. Global aggregation: Two modes (naive vs reconciled)

Key insight

Your PRD distinguishes:

- **Naive aggregation:** Sum of all nodes' local_value (can be wrong after partial loss).
- **Reconciled global:** Derived from coordinator's authoritative operation log.

You should implement **both** now, even if reconciliation logic comes in Phase 3.

Recommended pattern

In Simulation:

```
// Naive: sum of in-memory node counters
func (s *Simulation) NaiveGlobalValue() int64 {
    s.mu.RLock()
    defer s.mu.RUnlock()
    var sum int64
    for _, node := range s.Nodes {
        sum += node.LocalValue()
    }
    return sum
}

// Authoritative: sum from coordinator DB
func (s *Simulation) AuthoritativeGlobalValue() (int64, error) {
    var sum sql.NullInt64
    err := s.CoordDB.QueryRow(`SELECT SUM(amount) FROM counter_operations`).Scan(&sum)
    if err != nil {
        return 0, err
    }
    if !sum.Valid {
        return 0, nil
    }
    return sum.Int64, nil
}
```

Why:

- **Demo value:** You can show the wrong number (naive) vs the correct number (authoritative) side-by-side.
- **Invariant I5:** “Global count is reproducible” — recompute from durable ops and compare.
- **Prepares for reconciliation:** You already have the authoritative sum; reconciliation just needs to repair node-local state to match.

6. Epoch management: Explicit epoch table

Recommended schema

```
CREATE TABLE node_epochs (
    node_id TEXT NOT NULL,
    epoch INTEGER NOT NULL,
    started_at TEXT NOT NULL,
    PRIMARY KEY (node_id, epoch)
);
```

Functions:

```

func InsertEpoch(db *sql.DB, nodeID string, epoch int64) error {
    _, err := db.Exec(`
        INSERT INTO node_epochs (node_id, epoch, started_at)
        VALUES (?, ?, ?)
    `, nodeID, epoch, time.Now().Format(time.RFC3339))
    return err
}

func GetLatestEpoch(db *sql.DB, nodeID string) (int64, error) {
    var epoch int64
    err := db.QueryRow(`
        SELECT MAX(epoch) FROM node_epochs WHERE node_id = ?
    `, nodeID).Scan(&epoch)
    if err == sql.ErrNoRows {
        return 0, nil // No epoch yet
    }
    return epoch, err
}

```

Usage:

- On `NewNode`:
 - Call `GetLatestEpoch`.
 - If this is a fresh start (no existing ops), use epoch 1.
 - If recovering from crash, use `max_epoch + 1`.
- Store the epoch in the `Node` struct; include it in every operation.

7. Summary of recommended changes to your Phase 2 plan

Area	Your current plan	Recommended refinement	Rationale
Operation insert	INSERT OR IGNORE (good)	Keep this; explicitly avoid INSERT OR REPLACE or upsert for events	Append-only event log; duplicates ignored, never mutated [1] [6] [10]
Dual-write	Write to node DB + coordinator DB in <code>ApplyDelta</code>	Add lightweight outbox pattern (or at least document failure mode); treat coordinator as authoritative	Avoid silent inconsistency; reconciliation repairs gaps [14] [13]
*sql.DB usage	One per node + coordinator (good)	Tune pools: <code>MaxOpenConns=1</code> for node DBs, small pool for coordinator; share *sql.DB across goroutines [15] [16]	SQLite write serialization; efficient connection reuse
Sequence generation	In-memory counter (good)	Persist <code>MAX(sequence)</code> in DB; recover on restart; guard with mutex	Monotonicity + crash recovery
Global aggregation	Sum of node local values	Add both <code>NaiveGlobalValue()</code> and <code>AuthoritativeGlobalValue()</code> (from coordinator)	Show wrong vs right; prep for reconciliation; invariant I5

Area	Your current plan	Recommended refinement	Rationale
Epoch management	node_epochs table (good)	Keep; use MAX (epoch) on restart; increment for new incarnation	Clear node incarnation tracking

8. Concrete code snippets for key functions

InsertOperation (**append-only**, INSERT OR IGNORE)

```
func InsertOperation(db *sql.DB, op model.Operation) (inserted bool, err error) {
    result, err := db.Exec(`
        INSERT OR IGNORE INTO counter_operations (
            operation_id, counter_id, node_id, epoch, sequence, amount, created_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?)
    `, op.OperationID, op.CounterID, op.NodeID, op.Epoch, op.Sequence, op.Amount, op

    if err != nil {
        return false, err
    }

    // Check if row was inserted (vs ignored)
    changes, err := result.RowsAffected()
    if err != nil {
        return false, err
    }

    return changes > 0, nil
}
```

NaiveGlobalValue **VS** AuthoritativeGlobalValue

```
func (s *Simulation) NaiveGlobalValue() int64 {
    s.mu.RLock()
    defer s.mu.RUnlock()
    var sum int64
    for _, node := range s.Nodes {
        sum += node.LocalValue()
    }
    return sum
}

func (s *Simulation) AuthoritativeGlobalValue() (int64, error) {
    var sum sql.NullInt64
    err := s.CoordDB.QueryRow(`SELECT SUM(amount) FROM counter_operations`).Scan(&sum)
    if err != nil {
        return 0, err
    }
    if !sum.Valid {
        return 0, nil
    }
}
```

```
    return sum.Int64, nil
}
```

9. Test additions

Add these to your Phase 2 tests:

- **TestInsertDuplicateIgnored:** Insert same `OperationID` twice; second is ignored, no error.
- **TestNaiveVsAuthoritative:** After applying ops, `NaiveGlobalValue()` == `AuthoritativeGlobalValue()` (before any crash).
- **TestSequenceRecovery:** Restart a node (recreate `Node` struct); verify `nextSeq` is recovered from `MAX(sequence)`.

If you'd like, I can draft the full `db.go`, `node.go`, and `simulation.go` implementations with these patterns baked in.

✱✱

1. <https://coddy.tech/docs/sqlite/upsert>
2. https://runebook.dev/en/docs/sqlite/lang_createtable/primkeyconst
3. https://runebook.dev/en/docs/sqlite/lang_createtable/uniqueconst
4. <https://www.w3tutorials.net/blog/sqlite-upsert-update-or-insert/>
5. <https://www.w3tutorials.net/blog/sqlite-insert-or-replace-into-vs-update-where/>
6. <https://coddy.tech/docs/sqlite/insert>
7. https://runebook.dev/en/docs/sqlite/lang_createindex/uniqueidx
8. <https://coddy.tech/docs/sqlite/unique-constraints>
9. <https://www.bitsfolio.com/fix-python-sqlite3-duplicate-rows-every-script-run/>
10. <https://www.systemoverflow.com/learn/data-pipelines-orchestration/pipeline-idempotency/trade-offs-append-only-vs-idempotent-upserts>
11. <https://openillumi.com/en/en-sqlite-upsert-on-conflict-2/>
12. <https://www.hisqlboy.com/blog/mastering-upsert-on-conflict>
13. <https://aloknecessary.in/blogs/dual-write-problem-in-event-driven-architecture/>
14. <https://medium.com/threadsafe/exactly-once-processing-across-kafka-and-databases-using-the-outbox-pattern-f08fd640f683>
15. <https://learn.microsoft.com/en-us/sql/connect/golang/connection-pooling?view=sql-server-ver17>
16. <https://tanhdev.com/series/high-concurrency-systems/golang-database-connection-pool-optimization/>
17. <https://ask.csdn.net/questions/9094141>